Ankush Rane

raneankush93@gmail.com | github.com/kara-ops

Education

Western College of Commerce and Business Management Mumbai, India
B.Sc. in Computer Science

Technical Skills

Languages: Python
Frameworks & Tools: FastAPI, SQLAlchemy (Asyncio), Alembic, Celery, pytest, Docker, Uvicorn, React, Vite, Tailwind CSS
Databases: PostgreSQL (pgvector), Redis

Projects

DocChat | *FastAPI, PostgreSQL/pgvector, Redis, Celery, React, Groq/Gemini*

- Built a multi-tenant RAG document workspace platform enabling authenticated users to create workspaces, invite collaborators with role-based access control (owner/admin/member/viewer), and query uploaded documents via LLM-backed retrieval.
- Implemented hybrid retrieval combining pgvector cosine similarity with BM25 keyword ranking, merged via Reciprocal Rank Fusion (RRF), with numbered source citations grounding each AI response.
- Designed an asynchronous ingestion pipeline using Celery to process PDF/DOCX/TXT uploads, decoupling document upload from indexing for a non-blocking workspace experience.
- Extended the auth layer with session binding and device/location metadata collection, applying Redis-backed risk scoring to flag anomalous session behavior.
- Built a React, Vite, and Tailwind CSS frontend consuming the FastAPI backend, delivering workspace, document, and chat flows behind Google OAuth and JWT session handling.
- Applied Redis-backed rate limiting (token bucket, fixed window, sliding window) across auth and query endpoints to mitigate abuse and brute-force access patterns.

Standalone Identity Provider | *FastAPI, PostgreSQL, Redis, JWT*

- Built a decoupled authentication microservice handling centralized user identity, dual-token JWT/refresh-token sessions, and Google OAuth 2.0 alongside local email/password login.
- Diagnosed and fixed a refresh-token race condition affecting concurrent SPA requests by introducing a Redis-polling-based grace period, eliminating unintended logouts.
- Improved read-endpoint throughput from 60–70 req/s to 100+ req/s by adding a Redis caching layer in front of PostgreSQL.
- Migrated blocking PostgreSQL queries to SQLAlchemy's async engine (AsyncSession), removing event-loop blocking under concurrent load.
- Load-tested the service with Locust, sustaining 100+ requests/sec across 6,200+ requests with a 100% success rate on login and signup; identified bcrypt password hashing as the CPU-bound bottleneck versus the I/O-bound Redis-backed refresh path.
- Implemented Redis-backed rate limiting on authentication endpoints and CSRF protection on OAuth callbacks via state validation and secure cookie policies.

Rate Limiter Microservice | *FastAPI, Redis, Lua*

- Designed a rate limiting microservice implementing three algorithms — Fixed Window, Token Bucket, and Sliding Window — exposed as reusable, namespaced FastAPI dependencies.
- Eliminated race conditions under concurrent access by executing rate-limit checks atomically, using custom Lua scripts for Token Bucket and Sliding Window and Redis pipelines for Fixed Window.
- Applied algorithm selection by endpoint sensitivity: Fixed Window for high-volume reads, Token Bucket for write/mutate operations, and Sliding Window Log for auth and information-leak-prone endpoints requiring precise limits.
- Built proxy-aware client identification, parsing `X-Forwarded-For` headers with a fallback to direct connection IP for accurate limiting behind load balancers.
- Structured the service for distributed deployment, using `redis.asyncio` for a fully async, non-blocking request path across scaled FastAPI instances.